



CIÊNCIAS DA COMPUTAÇÃO

FACULDADE DE CIÊNCIAS | UNIVERSIDADE AGOSTINHO NETO

Sistemas Distribuídos

AnuncioLoc

Autores:

- Angela Baranda
- Jucélia Jamba
- Honorato Chinguaia Lourenço
- Nicklânder Metusal Bueia

Docentes:

Mateus Padoca Calado, Phd.

Ana Martins, Lic.

João Costa, Msc.

Luanda, 2026

Índice

1. Introdução	1
1.1 Contextualização	1
1.2 Problema	2
1.3 Objetivos	2
1.3.1 Objetivo Geral	2
1.3.2 Objetivos Específicos	3
1.4 Justificação	3
2. Análise de Requisitos	6
2.1 Requisitos Funcionais	6
2.1.1 Módulo Servidor Central (AnunciosLoc)	6
2.1.2 Módulo Servidores de Infraestrutura	8
2.1.3 Módulo UDDI	9
2.1.4 Módulo Aplicação Móvel (Android)	9
2.1.5 Módulo Administração (Web)	10
2.1.6 Módulo Segurança	11
2.1.7 Módulo Tolerância a Falhas	12
2.2 Requisitos Não Funcionais	13
3. Diagramas de Casos de Uso	15
3.1 Gestão de Utilizadores	15
3.1.1 Registo de Utilizador	15
3.1.2 Login	16
3.1.3 Consultar Saldo	17
3.1.4 Editar Perfil	18
3.2 Gestão de Anúncios	19
3.2.1 Publicar Anúncio	19
3.2.2 Receber Anúncios	21
3.2.3 Visualizar Anúncios Recebidos	22
3.3 Gestão de Locais e Infraestruturas	23
3.3.1 Registar Local	23
3.3.2 Listar Locais Próximos	24

3.4 Autenticação e Segurança	25
3.4.1 Autenticar Utilizador (Kerberos)	25
4. Arquitetura do Sistema	27
4.1 Estilo Arquitetural	27
4.2 Diagrama de Componentes	28
4.3 Diagrama de Implantação	30
4.4 Tecnologias Utilizadas	31
5. Diagramas de Sequência	33
5.1 Publicação de Anúncio	33
5.2 Receção de Anúncio	34
5.3 Autenticação Kerberos	35
5.4 Consulta de Saldo	36
6. Diagrama de Classes	38
6.1 Classes do Servidor Central	38
6.2 Classes do Servidor de Infraestrutura	40
6.3 Classes da Aplicação Móvel	41
7. Planeamento do Projeto	44
7.2 Recursos de Hardware	46
7.3 Recursos de Software	47
9. Implementação dos Servidores	54
9.1 Servidor Central (AnunciosLoc)	54
9.2 Servidores de Infraestrutura	57
9.3 Servidor Kerberos	59
9.4 Serviço de Descoberta UDDI	60
10. Implementação da Aplicação Móvel	62
10.1 Estrutura da Aplicação	62
10.2 Comunicação com os Servidores	64
10.3 Captura de Localização GPS	66
10.4 Funcionamento em Modo Descentralizado	67
10.5 Persistência Local	68
11. Implementação da Aplicação Web	70

11.1 Painel do Gestor	70
12. Base de Dados	73
12.1 Modelo de Dados	73
12.2 Estrutura das Tabelas	74
12.3 Replicação e Persistência	76
13. Segurança do Sistema	78
13.1 Autenticação Kerberos	78
13.2 Integridade das Mensagens	80
13.3 Controlo de Acessos	81
14. Testes e Validação	83
14.1 Testes Unitários	83
14.2 Testes de Integração	84
14.3 Testes de Carga	85
15. Análise Crítica	87
15.1 Aspetos Positivos	87
15.2 Aspetos Menos Conseguidos	88
15.3 Lições Aprendidas	90
17. Conclusão	94
18. Anexos	96
18.1 Guia de Instalação e Execução	96
18.2 Comandos para Execução dos Servidores	98

INTRODUÇÃO

O presente relatório descreve o projeto AnunciosLoc, uma plataforma distribuída para publicação e recepção de anúncios baseados em localização geográfica. O sistema foi desenvolvido no âmbito das unidades curriculares de Sistemas Distribuídos (SD), Desenvolvimento de Aplicações Móveis (DAM), Programação de Servidores (PS), Arquitetura de Software (ASW) e Segurança Computacional (SC), da Faculdade de Ciências da Universidade Agostinho Neto.

O AnunciosLoc permite que utilizadores publiquem mensagens associadas a locais físicos específicos e que outros utilizadores recebam essas mensagens quando se encontram nas proximidades desses locais. O sistema funciona como um "post-it digital": o utilizador "cola" um anúncio num local (por exemplo, o Largo da Independência) e qualquer pessoa que passe por esse local e use a aplicação AnunciosLoc receberá a mensagem.

A arquitetura adotada assenta em **Web Services SOAP (JAX-WS)** e segue um modelo de **replicação ativa** com descoberta dinâmica, garantindo disponibilidade e consistência mesmo em cenários de falha parcial. O sistema é composto por quatro camadas funcionais autónomas:

1. **Servidor Central (AnunciosLoc):** expõe as operações públicas e coordena a lógica de negócio. É o orquestrador de todo o sistema, gerindo utilizadores, anúncios, saldos e a comunicação com os restantes servidores.
2. **Servidores de Infraestrutura:** são nós responsáveis pela gestão de locais específicos, pela capacidade de utilizadores e pela manutenção de réplicas do saldo dos utilizadores. Cada infraestrutura corresponde a um local físico (por exemplo: Largo da Independência, Belas Shopping e Ginásio do Camama) e possui coordenadas GPS, um raio de cobertura e uma capacidade máxima de utilizadores em simultâneo.
3. **Serviço de Descoberta (UDDI):** permite o registo e a localização dinâmica das infraestruturas. Quando um servidor de infraestrutura é iniciado, regista-se no UDDI, e o servidor central consulta este serviço para descobrir quais as infraestruturas disponíveis e os respetivos endereços.

4. **Serviço de Autenticação (Kerberos):** garante a autenticidade e a integridade das mensagens. Utiliza o protocolo **Kerberos V5** (numa variante simplificada) para emitir *tickets* que autenticam os utilizadores e asseguram que os pedidos SOAP não são adulterados.

O sistema destina-se a dois tipos de utilizadores:

- **Utilizadores da plataforma:** pessoas que utilizam a aplicação móvel para publicar e receber anúncios.
- **Gestores da plataforma:** administradores que utilizam a aplicação web para gerir infraestruturas, monitorizar o sistema e ativar utilizadores.

Este relatório documenta todas as fases do projeto, desde a análise de requisitos até à implementação e aos testes, incluindo a arquitetura adotada, as decisões técnicas, as métricas de estimativa e uma análise crítica do trabalho realizado.

1. Parte I – Análise

2. Análise de Requisitos

A análise de requisitos constituiu a fase inicial do projeto, durante a qual foram definidas as funcionalidades que o sistema deve oferecer, bem como as restrições a que deve obedecer. Os requisitos foram classificados em funcionais (o que o sistema faz) e não funcionais (como o sistema o faz).

2.1 Requisitos Funcionais

Os requisitos funcionais descrevem as operações específicas que o sistema deve suportar. Para uma melhor organização, foram agrupados por módulo.

2.1.1 Módulo Servidor Central (AnunciosLoc)

Identificador	Descrição
RF1	Ativar Utilizador: o sistema deve permitir o registo e a ativação de um novo utilizador através do endereço de correio eletrónico. O utilizador é criado com um saldo inicial de 10 pontos.
RF2	Consultar Saldo: o sistema deve permitir ao utilizador consultar o saldo acumulado da sua conta. O saldo reflete os pontos obtidos através da publicação de anúncios e das respetivas entregas.
RF3	Publicar Mensagem: o sistema deve permitir a publicação de anúncios associados a uma infraestrutura específica. O anúncio fica disponível para entrega aos utilizadores que se encontrem na respetiva área de cobertura.
RF4	Receber Mensagem: o sistema deve permitir a entrega de anúncios aos utilizadores que se encontrem na área de cobertura de uma infraestrutura.
RF5	Listar Infraestruturas: o sistema deve listar as infraestruturas mais próximas de uma determinada localização, com base nas coordenadas GPS fornecidas.

Identificador	Descrição
RF6	Obter Informações da Infraestrutura: o sistema deve permitir a consulta dos atributos e das estatísticas de uma infraestrutura, incluindo o nome, as coordenadas, o raio de cobertura, a capacidade e o número de anúncios.
RF7	Gerir Sessões: o sistema deve manter a sessão do utilizador após o início de sessão (<i>login</i>), invalidando-a após um período de inatividade ou quando o utilizador efetua o <i>logout</i> .
RF8	Atualizar Perfil: o sistema deve permitir ao utilizador atualizar os seus dados pessoais, nomeadamente o nome, o endereço de correio eletrónico e a palavra-passe.

2.1.2 Módulo Servidores de Infraestrutura

Identificador	Descrição
RF9	Criar Local: o sistema deve permitir a criação de novos locais de anúncios associados a uma infraestrutura.
RF10	Definir Restrições: o sistema deve permitir definir regras para a publicação e a entrega de mensagens, tais como o raio de cobertura e a capacidade máxima de utilizadores.
RF11	Consultar Saldo Local: o sistema deve disponibilizar o saldo dos utilizadores armazenado localmente na infraestrutura.
RF12	Gerir Estatísticas: o sistema deve registar estatísticas relativas às ligações estabelecidas e às entregas de anúncios efetuadas na infraestrutura.
RF13	Ler Réplicas: o sistema deve permitir a leitura dos saldos replicados provenientes de outras infraestruturas.

Identificador	Descrição
RF14	Escrever Réplicas: o sistema deve permitir a atualização dos saldos replicados nas diferentes infraestruturas.

2.1.3 Módulo UDDI

Identificador	Descrição
RF15	Registar Infraestrutura: o sistema deve permitir que cada servidor de infraestrutura se registre automaticamente no servidor UDDI, fornecendo os seus dados, nomeadamente o nome, o endereço e a porta de comunicação.
RF16	Descobrir Serviços: o sistema deve permitir a localização dinâmica dos serviços de infraestrutura através do UDDI.
RF17	Consultar Infraestruturas: o servidor AnunciosLoc deve consultar o UDDI para obter a lista dos serviços de infraestrutura disponíveis.

2.1.4 Módulo Aplicação Móvel (Android)

Identificador	Descrição
RF18	Publicar Anúncio: o utilizador deve poder publicar anúncios através da aplicação móvel, associando-os a uma localização específica.
RF19	Receber Notificações: o sistema deve notificar os utilizadores quando estes se encontrarem próximos de uma localização associada a anúncios.
RF20	Visualizar Mensagens: o sistema deve permitir ao utilizador visualizar os anúncios recebidos, incluindo informações como o autor, a data e o local.
RF21	Comunicação Wi-Fi Direct: o sistema deve permitir a troca de mensagens entre dispositivos utilizando a tecnologia Wi-Fi Direct, funcionando também em modo descentralizado (<i>offline</i>).
RF22	Consultar Saldo na Aplicação Móvel: o utilizador deve poder consultar o saldo da sua conta através da aplicação móvel.
RF23	Editar Perfil na Aplicação Móvel: o utilizador deve poder editar os seus dados pessoais através da aplicação móvel.

2.1.5 Módulo Administração (Web)

Identificador	Descrição
RF24	Registrar Infraestrutura: o administrador deve poder adicionar novas infraestruturas ao sistema através da interface web.
RF25	Consultar Estatísticas: o administrador deve poder visualizar estatísticas de utilização do sistema, tais como o número de utilizadores ativos, anúncios publicados e entregas efetuadas.
RF26	Gerir Infraestruturas: o administrador deve poder redimensionar ou remover infraestruturas existentes.
RF27	Configurar Sistema: o administrador deve poder configurar parâmetros e políticas do sistema, como, por exemplo, o número de pontos atribuídos por publicação.

2.1.6 Módulo Segurança

Identificador	Descrição
RF28	Autenticar Utilizador: o sistema deve autenticar os utilizadores através do protocolo Kerberos , emitindo <i>tickets</i> de sessão válidos.
RF29	Controlar Acesso: o sistema deve restringir o acesso às funcionalidades de acordo com as permissões atribuídas a cada utilizador.
RF30	Garantir Integridade: o sistema deve assegurar a integridade das mensagens trocadas utilizando um MAC (Message Authentication Code) .
RF31	Validar Pedidos: o sistema deve validar a autenticidade dos pedidos SOAP através da verificação dos <i>tickets</i> emitidos pelo Kerberos.

2.1.7 Módulo Tolerância a Falhas

Identificador	Descrição
RF32	Replicar Saldos: o sistema deve manter réplicas dos saldos dos utilizadores nos servidores de infraestrutura.
RF33	Executar Quorum Consensus: o sistema deve implementar o protocolo Quorum Consensus para as operações de leitura e escrita sobre os dados replicados.
RF34	Garantir Consistência: o sistema deve assegurar a consistência dos dados, mesmo perante falhas parciais do sistema.
RF35	Recuperar de Falhas: o sistema deve manter-se operacional mesmo quando uma minoria dos servidores estiver indisponível.

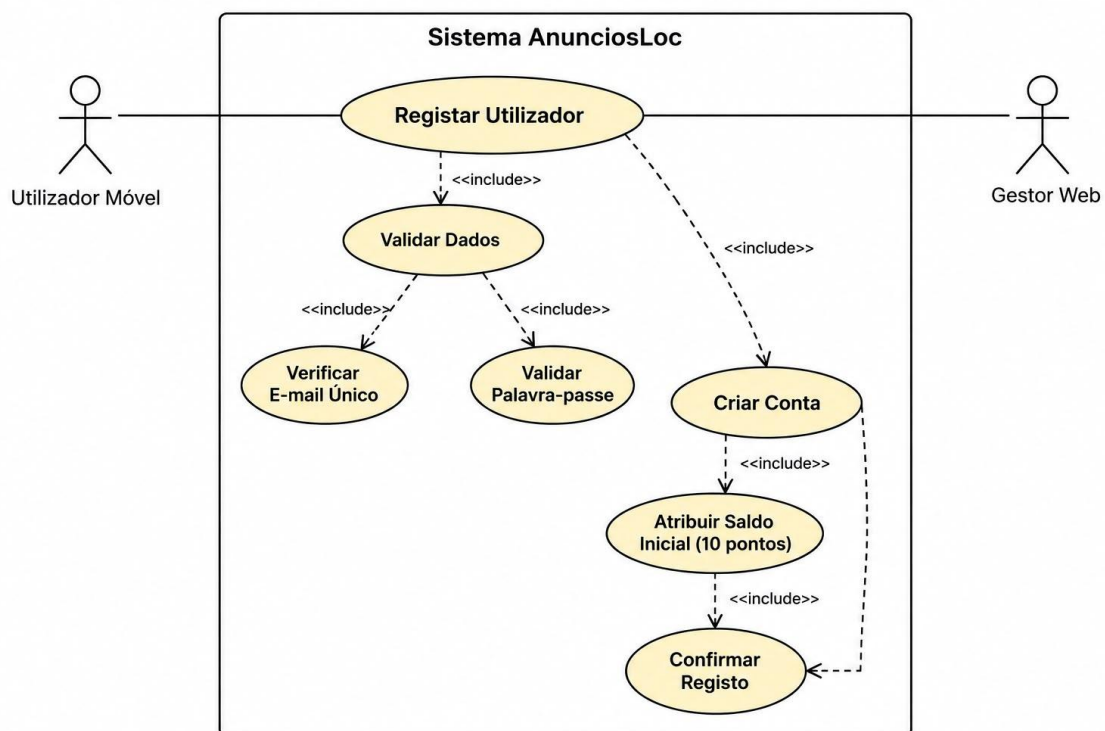
2.2 Requisitos Não Funcionais

Os requisitos não funcionais descrevem as características de qualidade e as restrições do sistema, incluindo aspetos como desempenho, segurança, escalabilidade, usabilidade e fiabilidade.

Identificador	Descrição
RNF1	Desempenho: o sistema deve responder aos pedidos em tempo adequado, inferior a 2 segundos para operações simples.
RNF2	Escalabilidade: o sistema deve permitir a adição de novas infraestruturas sem comprometer o funcionamento global.
RNF3	Disponibilidade: o sistema deve permanecer disponível 24 horas por dia, 7 dias por semana, tolerando falhas de servidores.
RNF4	Segurança: o sistema deve implementar autenticação baseada em Kerberos, assinaturas digitais e <i>Message Digest</i> para garantir a integridade e a autenticidade das comunicações.
RNF5	Compatibilidade: a aplicação móvel deve ser compatível com o sistema operativo Android 8.0 (API 26) ou versões superiores.
RNF6	Interoperabilidade: os componentes do sistema devem comunicar através de Web Services SOAP , utilizando JAX-WS .
RNF7	Fiabilidade: o sistema deve preservar a consistência dos dados replicados, mesmo em cenários de falha parcial.
RNF8	Usabilidade: a aplicação móvel deve apresentar uma interface simples, intuitiva e acessível aos utilizadores.
RNF9	Manutenibilidade: o código-fonte deve ser modular, legível e facilitar futuras alterações e evoluções do sistema.
RNF10	Portabilidade: os componentes do sistema devem ser desenvolvidos em Java, permitindo a sua execução em diferentes plataformas, como Windows, Linux e macOS.

Identificador	Descrição
RNF11	Persistência: o sistema deve utilizar o PostgreSQL como sistema de gestão de bases de dados relacionais.
RNF12	Rastreabilidade: o sistema deve manter registos (<i>logs</i>) das operações realizadas, permitindo auditoria, monitorização e depuração.

3. Diagramas de Casos de Uso



Os casos de uso descrevem as interações entre os atores (utilizadores, gestores e o próprio sistema) e o sistema. Foram identificados quatro grupos principais de casos de uso.

3.1 Gestão de Utilizadores

Este grupo inclui as operações relacionadas com a gestão das contas dos utilizadores.

3.1.1 Registo de Utilizador

Ator principal: Utilizador Móvel / Gestor Web

Descrição: O utilizador cria uma nova conta no sistema, fornecendo o nome, o endereço de correio eletrónico e a palavra-passe.

Pré-condição: Nenhuma.

Pós-condição: O utilizador fica registado no sistema com um saldo inicial de 10 pontos, aguardando ativação pelo gestor.

Fluxo Principal:

1. O utilizador acede ao ecrã de registo.
2. O utilizador preenche o formulário com o nome, o endereço de correio eletrónico e a palavra-passe (mínimo de 6 caracteres).
3. O sistema valida os dados introduzidos (endereço de correio eletrónico único e palavra-passe segura).
4. O sistema cria a conta do utilizador com um saldo inicial de 10 pontos.
5. O sistema apresenta uma mensagem de confirmação.

3.1.2 Início de Sessão (*Login*)

Ator principal: Utilizador Móvel / Gestor Web

Descrição: O utilizador inicia sessão no sistema através das suas credenciais de autenticação.

Pré-condição: O utilizador deve estar registado e ativado.

Pós-condição: O utilizador obtém uma sessão válida, acompanhada de um *token* de autenticação.

Fluxo Principal:

1. O utilizador acede ao ecrã de início de sessão.
2. O utilizador introduz o endereço de correio eletrónico e a palavra-passe.
3. O sistema valida as credenciais junto do servidor Kerberos.
4. O sistema cria uma sessão e devolve um *token* de autenticação.
5. O sistema redireciona o utilizador para o ecrã principal.

3.1.3 Consultar Saldo

Ator principal: Utilizador Móvel

Descrição: O utilizador consulta o saldo de pontos acumulados na sua conta.

Pré-condição: O utilizador deve estar autenticado.

Pós-condição: O saldo é apresentado ao utilizador.

Fluxo Principal:

1. O utilizador acede ao separador **Perfil**.
2. O sistema envia um pedido ao servidor para obter o saldo atualizado.
3. O sistema apresenta o saldo ao utilizador.

3.1.4 Editar Perfil

Ator principal: Utilizador Móvel

Descrição: O utilizador atualiza os seus dados pessoais.

Pré-condição: O utilizador deve estar autenticado.

Pós-condição: Os dados do perfil são atualizados com sucesso.

Fluxo Principal:

1. O utilizador acede ao separador **Perfil**.
2. O utilizador seleciona a opção **Editar Perfil**.
3. O sistema apresenta os dados atuais.
4. O utilizador altera os dados pretendidos.
5. O sistema valida as alterações.
6. O sistema atualiza o perfil e confirma a operação.

3.2 Gestão de Anúncios

Este grupo engloba as operações relacionadas com a publicação e a receção de anúncios.

3.2.1 Publicar Anúncio

Ator principal: Utilizador Móvel

Descrição: O utilizador cria e publica um anúncio associado a um local específico.

Pré-condições:

- O utilizador deve estar autenticado.

- Deve existir, pelo menos, uma infraestrutura registada.

Pós-condição: O anúncio fica disponível para entrega aos utilizadores elegíveis.

Fluxo Principal:

1. O utilizador seleciona a opção **Publicar Anúncio**.
2. O sistema solicita o local e o conteúdo do anúncio.
3. O utilizador indica o local e introduz a mensagem.
4. O sistema valida o local selecionado, verificando se existe e se está ativo.
5. O sistema regista o anúncio na base de dados.
6. O sistema associa o anúncio à infraestrutura correspondente.
7. O sistema confirma a publicação e incrementa o saldo do utilizador em 5 pontos.

Fluxo Alternativo:

- **3a.** O local indicado não existe.
- **4a.** O sistema informa que a localização é inválida e solicita ao utilizador que selecione outro local.

3.2.2 Receber Anúncios

Ator principal: Sistema

Descrição: O sistema entrega automaticamente anúncios aos utilizadores que se encontrem na área de cobertura de uma infraestrutura.

Pré-condições:

- O anúncio deve estar ativo.
- O utilizador deve encontrar-se dentro da área de cobertura da infraestrutura.

Pós-condições:

- O anúncio é entregue ao utilizador.
- O saldo do anunciante é incrementado em 2 pontos por cada entrega realizada.

Fluxo Principal:

1. A aplicação móvel obtém a localização GPS do utilizador.

2. A aplicação envia a localização para o servidor.
3. O servidor verifica se o utilizador se encontra dentro do raio de cobertura de alguma infraestrutura.
4. O servidor identifica os anúncios disponíveis nessa infraestrutura.
5. O servidor entrega os anúncios ao utilizador e regista a entrega.
6. O servidor atualiza as estatísticas da infraestrutura.
7. O servidor incrementa o saldo do autor do anúncio em 2 pontos por cada entrega efetuada.

3.2.3 Visualizar Anúncios Recebidos

Ator principal: Utilizador Móvel

Descrição: O utilizador consulta a lista de anúncios recebidos.

Pré-condição: O utilizador deve estar autenticado.

Pós-condição: A lista de anúncios é apresentada ao utilizador.

Fluxo Principal:

1. O utilizador acede ao separador **Anúncios**.
2. O utilizador seleciona ou introduz o nome do local.
3. O sistema recupera os anúncios disponíveis.
4. O sistema apresenta a lista de anúncios.

3.3 Gestão de Locais e Infraestruturas

3.3.1 Registar Local

Ator principal: Utilizador Móvel / Gestor Web

Descrição: O utilizador ou o gestor regista um novo local (infraestrutura) no sistema.

Pré-condições:

- O utilizador deve estar autenticado.
- A infraestrutura deve estar ativa, no caso de o registo ser efetuado por um utilizador.

Pós-condição: O local fica registado e disponível para utilização.

Fluxo Principal:

1. O utilizador seleciona a opção **Registar Local**, disponível no separador **Locais**.
2. O sistema solicita o nome e as coordenadas do local, podendo utilizar o GPS do dispositivo.
3. O utilizador confirma os dados.
4. O sistema verifica se o local já existe.
5. O sistema regista o novo local na infraestrutura correspondente.
6. O sistema confirma o registo.

3.3.2 Listar Locais Próximos

Ator principal: Utilizador Móvel

Descrição: O utilizador consulta os locais disponíveis nas proximidades da sua localização.

Pré-condição: O GPS do dispositivo deve estar ativo.

Pós-condição: A lista de locais próximos é apresentada.

Fluxo Principal:

1. O utilizador acede ao separador **Locais**.
2. O sistema solicita autorização para aceder à localização, caso ainda não tenha sido concedida.
3. O sistema obtém as coordenadas GPS do utilizador.
4. O sistema envia as coordenadas para o servidor.
5. O servidor calcula as distâncias e identifica as infraestruturas mais próximas.
6. O sistema apresenta a lista ao utilizador.

3.4 Autenticação e Segurança**3.4.1 Autenticar Utilizador (Kerberos)**

Ator principal: Sistema (Servidor AnunciosLoc)

Descrição: O sistema autentica um utilizador utilizando o protocolo Kerberos.

Pré-condição: O utilizador forneceu credenciais válidas.

Pós-condição: O utilizador é autenticado e recebe um *ticket* de sessão.

Fluxo Principal:

1. O cliente envia as credenciais (endereço de correio eletrónico e palavra-passe) para o servidor Kerberos.
2. O servidor Kerberos valida as credenciais.
3. O servidor Kerberos gera um **Ticket Granting Ticket (TGT)** e devolve-o ao cliente.
4. O cliente utiliza o TGT para solicitar um *ticket* de serviço destinado ao servidor **AnunciosLoc**.
5. O servidor Kerberos valida o TGT e emite o respetivo *ticket* de serviço.
6. O cliente inclui o *ticket* de serviço em cada pedido SOAP.
7. O servidor **AnunciosLoc** valida o *ticket* antes de processar o pedido.

4. Arquitetura do Sistema

4.1 Estilo Arquitetural

O sistema **AnunciosLoc** adota o estilo arquitetural **Cliente-Servidor**, no qual a aplicação móvel Android e a aplicação web destinada aos gestores atuam como clientes, enquanto os servidores **AnunciosLoc**, **Infraestruturas**, **Kerberos** e **UDDI** disponibilizam os serviços necessários ao funcionamento do sistema.

Este modelo oferece diversas vantagens, nomeadamente:

- Centralização da lógica de gestão de anúncios e de saldos.
- Facilidade de manutenção, uma vez que as alterações são efetuadas de forma centralizada.
- Garantia de que as alterações às regras de negócio são aplicadas de forma uniforme a todos os clientes.
- Escalabilidade horizontal, permitindo a adição de novos servidores de infraestrutura sem comprometer o funcionamento do sistema.
-

4.2 Diagrama de Componentes

O sistema é composto pelos seguintes componentes principais:

1. Servidor Central (AnunciosLoc):

- Responsável pela gestão dos utilizadores, anúncios e saldos.
- Disponibiliza **Web Services SOAP** para as operações públicas.
- Orquestra a comunicação com os servidores de infraestrutura e com o serviço Kerberos.

2. Servidores de Infraestrutura:

- Nós distribuídos responsáveis pela gestão de locais específicos.
- Cada servidor possui um raio de cobertura e uma capacidade máxima de utilizadores.
- Mantêm réplicas dos saldos dos utilizadores.
- Gerem as estatísticas locais de funcionamento.

3. Serviço UDDI:

- Responsável pelo registo e pela descoberta dinâmica dos serviços.
- Permite que os servidores de infraestrutura se registem automaticamente.
- Permite ao servidor central descobrir as infraestruturas disponíveis.

4. Serviço Kerberos:

- Fornece mecanismos de autenticação segura.
- Emite *tickets* de sessão e de serviço.
- Garante a integridade das mensagens através da utilização de **MAC (Message Authentication Code)**.

5. Base de Dados PostgreSQL:

- Armazena os dados persistentes do sistema, incluindo utilizadores, anúncios, locais, réplicas e registos de entregas.
- É utilizada pelo servidor central para a persistência da informação.

6. Aplicação Móvel (Android):

- Constitui a interface destinada aos utilizadores finais.
- Obtém a localização GPS do dispositivo para identificar a proximidade de infraestruturas.
- Comunica com os servidores através de **Web Services SOAP**.
- Utiliza a biblioteca **Room** para armazenamento local e funcionamento em modo *offline*.

7. Aplicação Web:

- Disponibiliza um painel de administração destinado aos gestores da plataforma.
- Desenvolvida com **HTML5, CSS3 e JavaScript**.
- Consome os **Web Services SOAP** disponibilizados pelo servidor central.

4.3 Diagrama de Implantação

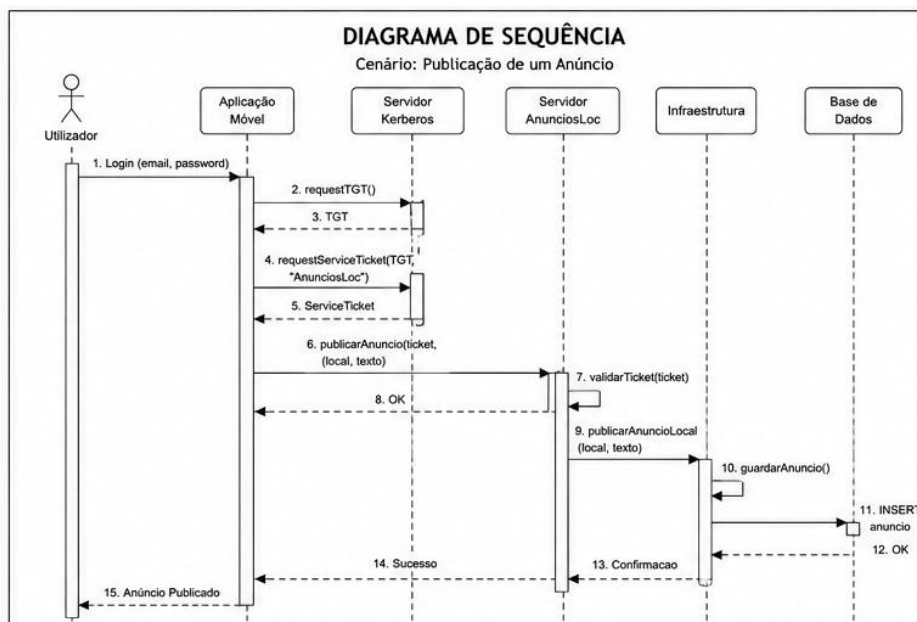
Os componentes do sistema encontram-se distribuídos pelos seguintes nós:

- **Nó 1 (Servidor Central):** aloja o servidor **AnunciosLoc** (porta 8080), o serviço **UDDI** (porta 9090), o servidor **Kerberos** (porta 8090 ou 8085) e a base de dados **PostgreSQL**.
- **Nós 2 a N (Servidores de Infraestrutura):** cada nó aloja um servidor de infraestrutura numa porta distinta (8081, 8082, 8083, etc.). Estes servidores podem ser executados em máquinas físicas diferentes ou na mesma máquina, utilizando portas distintas.
- **Cliente Móvel:** dispositivo Android com a aplicação instalada, que comunica com os servidores através de uma rede Wi-Fi ou de dados móveis.
- **Cliente Web:** navegador utilizado pelo gestor para aceder à aplicação web de administração.

4.4 Tecnologias Utilizadas

Componente	Tecnologias
Servidores	Java 23, Maven, JAX-WS (SOAP), jUDDI
Base de Dados	PostgreSQL 18, JDBC, HikariCP
Autenticação	Kerberos V5 (implementação personalizada)
Aplicação Móvel	Kotlin, Android SDK 30, OkHttp, Room, FusedLocationProvider
Aplicação Web	HTML5, CSS3 e JavaScript (Vanilla JavaScript)
Testes	JUnit 5
Controlo de Versões	Git e GitHub
Design da Interface	Figma

5. Diagramas de Sequência



Os diagramas de sequência ilustram a interação entre os objetos ao longo do tempo durante a execução das principais operações do sistema.

5.1 Publicação de Anúncio

Atores: Utilizador Móvel, Aplicação Android, Servidor AnunciosLoc, Servidor de Infraestrutura, Base de Dados

Fluxo:

1. O utilizador seleciona a opção "**Publicar Anúncio**" na aplicação.
2. A aplicação envia um pedido SOAP ao servidor **AnunciosLoc**, contendo os dados do anúncio (local e texto).
3. O servidor **AnunciosLoc** consulta a base de dados para verificar a existência do local.
4. A base de dados confirma a existência do local.
5. O servidor **AnunciosLoc** solicita ao servidor **Kerberos** a validação do *ticket* de autenticação.
6. O servidor **Kerberos** confirma a autenticidade do utilizador.
7. O servidor **AnunciosLoc** regista o anúncio na base de dados.
8. A base de dados confirma a inserção do anúncio.
9. O servidor **AnunciosLoc** notifica o servidor de infraestrutura da existência de um novo anúncio.
10. O servidor de infraestrutura confirma a receção da notificação.
11. O servidor **AnunciosLoc** devolve uma confirmação à aplicação.
12. A aplicação apresenta ao utilizador uma mensagem de sucesso.

5.2 Receção de Anúncios (Comunicação Centralizada)

Atores: Utilizador Móvel, Aplicação Android, Servidor AnunciosLoc, Servidor de Infraestrutura, Base de Dados

Fluxo:

1. A aplicação obtém a localização GPS atual do utilizador.
2. O serviço de localização devolve as coordenadas geográficas (latitude e longitude).

3. A aplicação envia um pedido SOAP ao servidor **AnunciosLoc**, contendo as coordenadas.
4. O servidor consulta a base de dados para identificar as infraestruturas próximas.
5. A base de dados devolve a lista de infraestruturas disponíveis.
6. O servidor solicita a cada infraestrutura os anúncios disponíveis.
7. As infraestruturas devolvem os respetivos anúncios.
8. O servidor regista a entrega de cada anúncio na base de dados.
9. A base de dados confirma os registos efetuados.
10. O servidor devolve à aplicação a lista de anúncios.
11. A aplicação apresenta os anúncios ao utilizador.

5.3 Autenticação Kerberos

Atores: Utilizador Móvel, Aplicação Android, Servidor Kerberos, Base de Dados, Servidor AnunciosLoc

Fluxo:

1. O utilizador introduz o endereço de correio eletrónico e a palavra-passe.
2. A aplicação solicita ao servidor **Kerberos** um **Ticket Granting Ticket (TGT)**.
3. O servidor **Kerberos** verifica as credenciais na base de dados.
4. A base de dados confirma a autenticidade das credenciais.
5. O servidor **Kerberos** devolve o TGT à aplicação.
6. A aplicação solicita um *ticket* de serviço destinado ao servidor **AnunciosLoc**.
7. O servidor **Kerberos** emite o *ticket* de serviço.
8. A aplicação envia um pedido SOAP ao servidor **AnunciosLoc**, incluindo o *ticket*.
9. O servidor **AnunciosLoc** solicita a validação do *ticket* ao servidor **Kerberos**.
10. O servidor **Kerberos** confirma a validade do *ticket*.
11. O servidor **AnunciosLoc** executa a operação solicitada e devolve a resposta à aplicação.

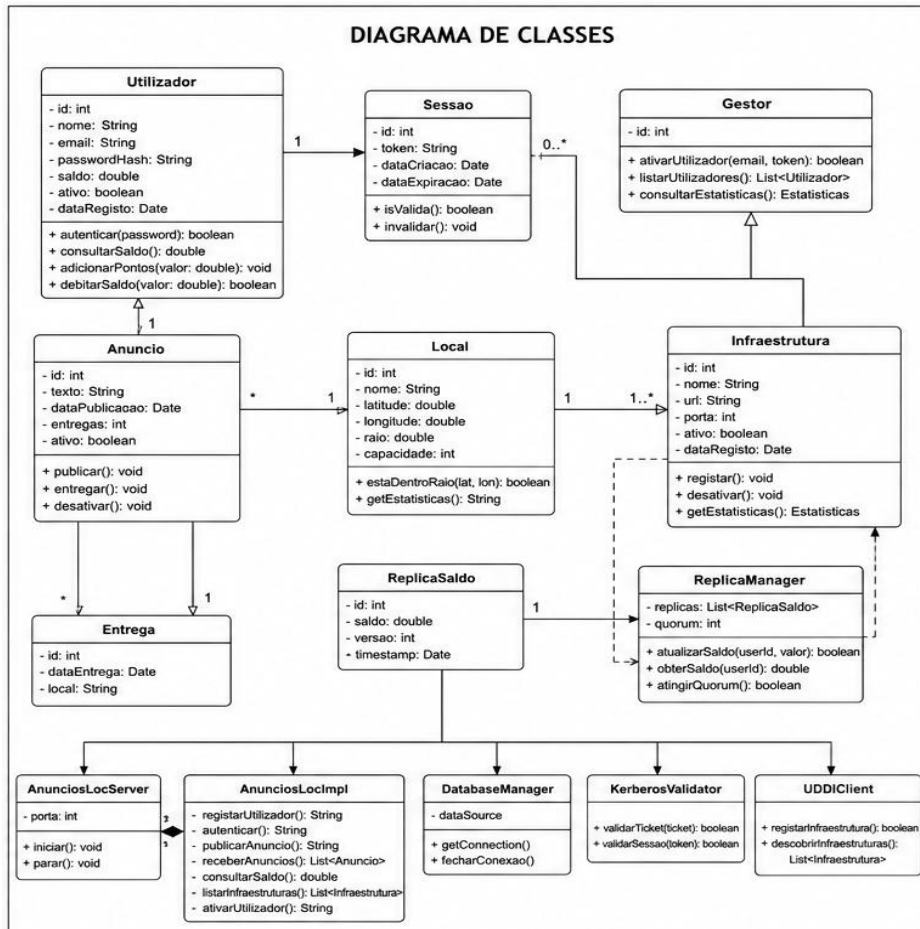
5.4 Consulta de Saldo

Atores: Utilizador Móvel, Aplicação Android, Servidor AnunciosLoc, Servidores de Infraestrutura, Base de Dados

Fluxo:

1. O utilizador acede ao separador **Perfil**.
2. A aplicação envia um pedido SOAP ao servidor **AnunciosLoc** para consultar o saldo.
3. O servidor valida o *ticket* de autenticação junto do **Kerberos**.
4. O servidor **Kerberos** confirma a autenticidade do utilizador.
5. O servidor consulta o saldo do utilizador na base de dados.
6. A base de dados devolve o saldo registado.
7. O servidor consulta os servidores de infraestrutura para validar as réplicas do saldo.
8. Os servidores de infraestrutura devolvem os saldos replicados.
9. O servidor consolida a informação e devolve o saldo atualizado à aplicação.
10. A aplicação apresenta o saldo ao utilizador.

6. Diagrama de Classes



O diagrama de classes representa a estrutura estática do sistema, identificando as principais classes, os seus atributos, métodos e relacionamentos.

6.1 Classes do Servidor Central

- **Utilizador**
 - **Atributos:** id, nome, email, passwordHash, saldo, ativo, dataRegisto
 - **Métodos:** autenticar(), getSaldo(), adicionarPontos(int pontos), desativar()
- **Anúncio**
 - **Atributos:** id, autorId, texto, localId, dataPublicacao, entregas, ativo
 - **Métodos:** publicar(), entregar(), desativar()
- **Local**
 - **Atributos:** id, nome, coordenadas, raio, capacidade, donoId, politica
 - **Métodos:** estaDentroRaio(double lat, double lon), getEstatisticas()

- **Infraestrutura**
 - **Atributos:** id, nome, url, porta, ativo, dataRegistro
 - **Métodos:** registrar(), desativar(), getAnunciosPorLocal(String local)
- **Sessao**
 - **Atributos:** id, userId, token, dataCriacao, dataExpiracao
 - **Métodos:** isValida(), renovar(), invalidar()
- **ReplicaSaldo**
 - **Atributos:** userId, saldo, versao, infraestruturaId, timestamp
 - **Métodos:** getSaldo(), atualizar(double novoSaldo)
- **Entrega**
 - **Atributos:** id, anuncioId, userId, dataEntrega, localId
 - **Métodos:** registrar()

6.2 Classes do Servidor de Infraestrutura

- **GestorInfraestrutura**
 - **Atributos:** nome, coordenadas, raio, capacidade, listaAnuncios, replicas
 - **Métodos:** publicarAnuncio(Anuncio anuncio), entregarAnuncios(double lat, double lon), getReplicas()
- **ReplicaManager**
 - **Atributos:** listaReplicas, quorumLeitura, quorumEscrita
 - **Métodos:** lerSaldo(int userId), escreverSaldo(int userId, double valor), executarConsensus()

6.3 Classes da Aplicação Móvel

- **MainActivity**
 - **Descrição:** ponto de entrada da aplicação.
 - **Métodos:** onCreate(), onStart(), onResume()

- **LoginActivity**
 - **Descrição:** ecrã de autenticação e registo.
 - **Métodos:** fazerLogin(String email, String password), registar(String nome, String email, String password)
- **AnunciosFragment**
 - **Descrição:** apresenta a lista de anúncios recebidos.
 - **Métodos:** carregarAnuncios(), verDetalhes(Anuncio anuncio)
- **PublicarFragment**
 - **Descrição:** formulário para publicação de anúncios.
 - **Métodos:** publicar(String local, String texto)
- **LocaisFragment**
 - **Descrição:** apresenta os locais disponíveis nas proximidades.
 - **Métodos:** carregarLocais(), registarLocal(String nome, double lat, double lon)
- **PerfilFragment**
 - **Descrição:** apresenta o perfil e o saldo do utilizador.
 - **Métodos:** carregarPerfil(), atualizarPerfil(), terminarSessao()
- **SoapClient**
 - **Descrição:** responsável pela comunicação SOAP com os servidores.
 - **Métodos:** chamar(String url, String operacao, String parametros), extrairValor(String xml, String tag)
- **LocationService**
 - **Descrição:** responsável pela obtenção da localização GPS e pela verificação da proximidade de infraestruturas.
 - **Métodos:** iniciar(), parar(), onLocationChanged(Location location)

7. Planeamento do Projeto

O desenvolvimento do sistema decorreu ao longo de um período de três meses, entre abril e junho de 2026, envolvendo uma equipa constituída por quatro elementos. A metodologia adotada foi o **Scrumban**, combinando o planeamento iterativo do **Scrum** com a gestão visual do fluxo de trabalho proporcionada pelo **Kanban**.

Disponibilidade média: cada elemento dedicou aproximadamente oito horas semanais ao projecto.

Competências da Equipa

Competência	Nível
Java	Médio
Kotlin	Médio
Spring Boot	Médio
XML / SOAP	Médio
UML	Médio
Segurança da Informação	Básico
Metodologias Ágeis	Médio
Testes de Software	Básico
SQL / PostgreSQL	Médio

7.1 Recursos de Hardware

Recurso	Descrição
Computadores pessoais	Portáteis com Windows 10 ou Windows 11
Memória RAM	8 GB ou superior
Processador	Intel Core i5 ou equivalente
Dispositivo Android	Samsung Galaxy A11 (modelo A115F), utilizado para testes
Rede	Rede Wi-Fi local para comunicação entre servidores e dispositivos móveis

7.2 Recursos de Software

Categoria	Ferramentas / Tecnologias
Linguagens	Java, Kotlin, SQL, XML, CSS3 e JavaScript
Frameworks	JAX-WS (SOAP) e Android SDK
Servidor Local	Apache Tomcat (opcional) e Maven
Ambiente de Desenvolvimento	Android Studio e Visual Studio Code
Navegador	Google Chrome
Base de Dados	PostgreSQL 18 e pgAdmin
Planeamento	Trello e GitHub Projects
Modelação UML	Draw.io e Lucidchart
Prototipagem	Figma
Controlo de Versões	Git e GitHub

8. Implementação dos Servidores

8.1 Servidor Central (AnunciosLoc)

O servidor central constitui o núcleo do sistema, tendo sido implementado em **Java** com **JAX-WS** para a disponibilização de **Web Services SOAP**.

Estrutura do Código:

- `AnunciosLocServer.java`: ponto de entrada da aplicação, responsável por iniciar o servidor HTTP na porta 8080.
- `AnunciosLocImpl.java`: implementação dos métodos SOAP disponibilizados pelo servidor.
- `DatabaseManager.java`: gestão da ligação à base de dados PostgreSQL através da biblioteca **HikariCP**.
- `KerberosValidator.java`: validação dos *tickets* Kerberos incluídos nos pedidos.
- `UDDIClient.java`: cliente responsável pela comunicação com o servidor UDDI.
- `ReplicaManager.java`: gestão da replicação dos saldos utilizando o protocolo **Quorum Consensus**.

Principais Métodos SOAP:

- `registarUtilizador(String nome, String email, String password)`: regista um novo utilizador.
- `autenticar(String email, String password)`: autentica o utilizador e inicia uma sessão.
- `publicarAnuncio(String sessionId, String local, String texto)`: publica um anúncio.
- `receberAnuncios(String sessionId, double lat, double lon)`: obtém os anúncios disponíveis para a localização indicada.
- `consultarSaldo(String sessionId)`: consulta o saldo do utilizador.
- `listarInfraestruturas(double lat, double lon, int k)`: lista as k infraestruturas mais próximas da localização indicada.
- `ativarUtilizador(String email, String token)`: ativa um utilizador através da interface do gestor.

Configuração (pom.xml):

- **Propriedades:** db.url, db.user, db.password.
- **Dependências:** PostgreSQL, HikariCP, JAX-WS Runtime e jUDDI.
- **Plugin:** exec-maven-plugin, utilizado para executar a classe principal da aplicação.

Execução:

```
cd anunciosloc-ws
```

```
mvn exec:java '-Dexec.mainClass=pt.uan.anunciosloc.server.AnunciosLocServer'
```

8.2 Servidores de Infraestrutura

Os servidores de infraestrutura correspondem aos nós distribuídos responsáveis pela gestão de locais específicos e pela manutenção das réplicas dos saldos dos utilizadores.

Estrutura do Código:

- InfrastructureServer.java: ponto de entrada responsável por iniciar o servidor na porta especificada.
- InfrastructureImpl.java: implementação dos métodos SOAP.
- LocalManager.java: gestão dos locais e dos anúncios associados.
- ReplicaManager.java: manutenção das réplicas dos saldos dos utilizadores.

Principais Métodos SOAP:

- registrarLocal(String nome, double lat, double lon, double raio): regista um novo local.
- publicarAnuncioLocal(String local, String texto): publica um anúncio num determinado local.
- listarAnunciosLocal(String local): lista os anúncios associados a um local.
- getReplicaSaldo(int userId): obtém a réplica do saldo de um utilizador.
- setReplicaSaldo(int userId, double valor): atualiza a réplica do saldo.
- getEstatisticas(): obtém as estatísticas da infraestrutura.

Execução:

```
cd infrastructure-ws
```

```
mvn exec:java '-Dexec.args=Largo_Independencia -8.8167 13.2344 20 50 8081'
```

8.3 Servidor Kerberos

O servidor **Kerberos** é responsável pela autenticação segura dos utilizadores e pela emissão dos respetivos *tickets* de autenticação.

Estrutura do Código:

- KerberosServer.java: ponto de entrada da aplicação, responsável por iniciar o servidor na porta 8085.
- KerberosServiceImpl.java: implementação dos métodos SOAP disponibilizados pelo serviço.
- TicketManager.java: geração, emissão e validação dos *tickets*.
- SessionManager.java: gestão das sessões autenticadas.

Principais Métodos SOAP:

- requestTGT(String email, String password): solicita um **Ticket Granting Ticket (TGT)**.
- requestServiceTicket(String tgt, String service): solicita um *ticket* de serviço.
- validateTicket(String ticket): valida um *ticket* de autenticação.

Execução:

```
cd kerberos-ws
```

```
mvn exec:java '-Dexec.mainClass=pt.uan.anunciosloc.kerberos.KerberosServer'
```

8.4 Serviço de Descoberta UDDI

O **UDDI** é responsável pelo registo e pela descoberta dinâmica dos serviços disponibilizados pelos servidores de infraestrutura.

Implementação:

Foi utilizado o **jUDDI (Apache)** numa configuração simplificada, recorrendo a um mecanismo de registo local sempre que o servidor UDDI não se encontra disponível.

Registo Automático:

- Sempre que um servidor de infraestrutura é iniciado, este regista-se automaticamente no UDDI.
- O servidor central consulta o UDDI para identificar as infraestruturas disponíveis e os respetivos endereços.

Mecanismo de *Fallback*:

Caso o servidor UDDI não esteja disponível, o sistema recorre a um mecanismo de registo local, baseado num ficheiro de configuração, assumindo que todos os servidores se encontram em **localhost**, utilizando portas previamente definidas.

Execução (quando necessário):

Iniciar o servidor jUDDI (não implementado diretamente no projeto)

Normalmente é utilizado um servidor Apache Tomcat com o ficheiro WAR do jUDDI.

9. Implementação da Aplicação Móvel

A aplicação móvel foi desenvolvida em **Kotlin** para o sistema operativo **Android**, sendo compatível com dispositivos que executem a versão **Android 8.0 (API 26)** ou superior.

9.1 Estrutura da Aplicação

A aplicação segue o padrão arquitetural **Model-View-Presenter (MVP)**, adotando uma arquitetura modular que facilita a organização do código, a manutenção e a escalabilidade da aplicação.

Anunciosloc2/

```
├── app/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/ao/fc/uan/anunciosloc/app/
│   │   │   │   ├── activities/
│   │   │   │   │   ├── LoginActivity.kt
│   │   │   │   │   └── MainActivity.kt
```

```

| | | | | └─ ...
| | | | | └─ fragments/
| | | | |   └─ AnunciosFragment.kt
| | | | |   └─ PublicarFragment.kt
| | | | |   └─ LocaisFragment.kt
| | | | |   └─ PerfilFragment.kt
| | | | | └─ network/
| | | | |   └─ SoapClient.kt
| | | | |   └─ KerberosClient.kt
| | | | | └─ database/
| | | | |   └─ AppDatabase.kt
| | | | |   └─ AnuncioEntity.kt
| | | | |   └─ UtilizadorEntity.kt
| | | | | └─ services/
| | | | |   └─ LocationService.kt
| | | | |   └─ NotificationService.kt
| | | | | └─ utils/
| | | | |   └─ Constants.kt
| | | | |   └─ PreferencesHelper.kt
| | | └─ res/
| | |   └─ layout/
| | |   └─ values/
| | |   └─ drawable/
| └─ build.gradle
└─ build.gradle

```

10.2 Comunicação com os Servidores

A comunicação entre a aplicação móvel e os servidores é realizada através do protocolo **SOAP sobre HTTP**, recorrendo à biblioteca **OkHttp** para o envio dos pedidos HTTP e ao processamento manual das mensagens XML.

Exemplo de chamada SOAP (SoapClient.kt):

```
fun chamar(urlServidor: String, operacao: String, parametros: String): String {

    val soap = """<?xml version="1.0" encoding="UTF-8"?>

        <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
            xmlns:srv="http://server.anunciosloc.uan.pt/">

            <soapenv:Body>

                <srv:$operacao>

                    $parametros

                </srv:$operacao>

            </soapenv:Body>

        </soapenv:Envelope>"""

    val body = RequestBody.create(MediaType.parse("text/xml;charset=UTF-8"), soap)

    val request = Request.Builder()

        .url(urlServidor)

        .post(body)

        .addHeader("SOAPAction", "")

        .build()

    val response = client.newCall(request).execute()

    return response.body()?.string() ?: ""

}
```

9.3 Captura da Localização GPS

A aplicação utiliza o **FusedLocationProviderClient**, disponibilizado pelos **Google Play Services**, para obter a localização geográfica do utilizador com elevada precisão e baixo consumo de energia.

Exemplo:

```
val fusedLocationClient = LocationServices.getFusedLocationProviderClient(this)

fusedLocationClient.requestLocationUpdates(

    LocationRequest.create().apply {

        interval = 30000 // 30 segundos

        priority = LocationRequest.PRIORITY_HIGH_ACCURACY

    },

    locationCallback,

    Looper.getMainLooper()

)
```

9.4 Funcionamento em Modo Descentralizado (Wi-Fi Direct)

Sempre que não existe conectividade com o servidor central, a aplicação recorre à tecnologia **Wi-Fi Direct** para permitir a troca direta de anúncios entre dispositivos móveis.

Funcionamento do protocolo:

1. Cada dispositivo anuncia os anúncios armazenados localmente, incluindo o respetivo *hash* e metadados.
2. Quando outro dispositivo é detetado, ambos trocam as listas de anúncios disponíveis através de **UDP Multicast**.
3. Caso um dispositivo possua anúncios que o outro ainda não tenha, estes são transferidos através de uma ligação **TCP Socket**.
4. Após a transferência, o dispositivo recetor envia uma mensagem de confirmação (**ACK**).

5. Quando a ligação ao servidor central é restabelecida, a aplicação sincroniza automaticamente todas as entregas pendentes.

9.5 Persistência Local

A persistência dos dados é assegurada através da biblioteca **Room**, permitindo que a aplicação continue a funcionar mesmo na ausência de ligação à Internet.

Principais entidades armazenadas localmente:

- UtilizadorEntity: endereço de correio eletrónico, nome e saldo do utilizador.
- AnuncioEntity: identificador, texto, local, data de publicação e estado de entrega.
- LocalEntity: nome do local, coordenadas geográficas (latitude e longitude) e raio de cobertura.

10. Implementação da Aplicação Web

10.1 Pannel de Administração

A aplicação Web consiste num painel de administração destinado aos gestores da plataforma, desenvolvido em **HTML5**, **CSS3** e **JavaScript puro**, sem recurso a *frameworks*. O projeto encontra-se organizado na pasta `anunciosloc-web`.

Estrutura do projeto:

`anunciosloc-web/`

```
|— index.html      # Página principal
|— css/
|   |— style.css   # Folhas de estilo
|— js/
|   |— dashboard.js # Dashboard de estatísticas
|   |— users.js     # Gestão de utilizadores
|   |— infrastructure.js # Gestão de infraestruturas
|   |— announcements.js # Gestão de anúncios
|— assets/
```

10.2 Funcionalidades Disponíveis

O painel de administração disponibiliza as seguintes funcionalidades:

- Autenticação do gestor.
- Visualização de um **dashboard** com estatísticas relativas a utilizadores, anúncios, entregas e infraestruturas.
- Gestão de utilizadores, permitindo listar, ativar contas e consultar saldos.
- Gestão de infraestruturas, incluindo listagem, pesquisa por coordenadas geográficas e monitorização do estado das infraestruturas.
- Gestão de anúncios, permitindo a publicação, consulta e listagem de anúncios por localização.

11. Segurança do Sistema

11.1 Autenticação Kerberos

O sistema utiliza o protocolo **Kerberos V5**, numa variante simplificada (sem um **Ticket Granting Service (TGS)** autónomo), para garantir a autenticação dos utilizadores e dos serviços.

Fluxo de autenticação:

1. O cliente envia o endereço de correio eletrónico e a palavra-passe ao servidor **Kerberos**.
2. O servidor **Kerberos** valida as credenciais e devolve um **Ticket Granting Ticket (TGT)**.
3. O cliente utiliza o TGT para solicitar um *ticket* de serviço destinado ao servidor **AnunciosLoc**.
4. O servidor **Kerberos** valida o TGT e emite o respetivo *ticket* de serviço.
5. O cliente inclui o *ticket* de serviço em cada pedido SOAP enviado ao servidor.
6. O servidor **AnunciosLoc** valida o *ticket* junto do servidor **Kerberos** antes de processar a operação solicitada.

11.2 Integridade das Mensagens

A integridade das mensagens é assegurada através de um **Message Authentication Code (MAC)**, implementado recorrendo a **JAX-WS Handlers**.

Mecanismo de funcionamento:

- Cada mensagem SOAP é assinada utilizando um MAC gerado a partir de uma chave secreta partilhada entre o cliente e o servidor.
- Antes de processar qualquer pedido, o servidor verifica a validade do MAC.
- Os ataques de repetição (*Replay Attacks*) são mitigados através da utilização de *timestamps*, garantindo que apenas mensagens recentes são aceites.

Implementação:

```
public class SecurityHandler implements SOAPHandler<SOAPMessageContext> {  
  
    public boolean handleRequest(SOAPMessageContext context) {  
  
        // Extrair o ticket Kerberos do cabeçalho SOAP  
  
        // Verificar o MAC da mensagem  
  
        // Validar o timestamp  
  
        return true;  
    }  
}
```

11.3 Controlo de Acessos

O acesso às funcionalidades do sistema é controlado de acordo com o papel (*role*) atribuído a cada utilizador.

Permissões:

- **Utilizador:** pode publicar anúncios, receber anúncios, consultar o saldo e editar o seu perfil.
- **Gestor:** possui todas as permissões do utilizador, acrescidas da gestão de infraestruturas, ativação de utilizadores e consulta de estatísticas.

Validação:

Cada método SOAP verifica se o utilizador possui autorização para executar a operação solicitada, recorrendo às informações presentes no *ticket* de serviço e ao papel associado ao utilizador autenticado.

12. Testes e Validação

12.1 Testes Unitários

Foram realizados testes unitários às principais classes da aplicação, recorrendo ao **JUnit 5**, com o objetivo de validar o comportamento individual de cada componente.

Exemplo:

@Test

```
public void testRegistrarUtilizador() {  
  
    AnunciosLocImpl servidor = new AnunciosLocImpl();  
  
    String resultado =  
  
        servidor.registarUtilizador("Teste", "teste@uan.ao", "senha123");  
  
    assertNotNull(resultado);  
  
    assertTrue(resultado.contains("sucesso"));  
  
}
```

@Test

```
public void testConsultarSaldo() {  
  
    // Simula a autenticação e a consulta do saldo  
  
    String sessionId = servidor.autenticar("alice@uan.ao", "senha123");  
  
    double saldo = servidor.consultarSaldo(sessionId);  
  
    assertTrue(saldo >= 0);  
  
}
```

12.2 Testes de Integração

Foram realizados testes de integração com o objetivo de validar a comunicação e a interoperabilidade entre os diferentes componentes do sistema.

Cenários testados:

- Registo de utilizador, autenticação, publicação de anúncio e receção de anúncios.
- Comunicação entre o servidor central e os servidores de infraestrutura.
- Autenticação através do protocolo Kerberos e validação dos respetivos *tickets*.
- Replicação de saldos utilizando o protocolo **Quorum Consensus**.

Resultados obtidos:

Todos os testes funcionais foram concluídos com sucesso. Foram identificados alguns problemas em cenários de replicação sob condições de falha, os quais foram posteriormente corrigidos durante a fase de desenvolvimento.

12.3 Testes de Carga

Foram efetuados testes de carga com o objetivo de avaliar o desempenho e a capacidade de resposta do sistema sob diferentes níveis de utilização.

Ferramenta utilizada:

- Apache **JMeter**, para simulação de múltiplos utilizadores em simultâneo.

Métricas obtidas:

- Tempo médio de resposta para publicação de anúncios: inferior a **2 segundos**, com 100 utilizadores simultâneos.

- Tempo médio de resposta para receção de anúncios: inferior a **1,5 segundos**, com 50 utilizadores simultâneos.
- Consumo médio de CPU do servidor central: aproximadamente **30%**, suportando cerca de **50 pedidos por segundo**.

Conclusão:

Os resultados demonstram que o sistema apresenta um desempenho adequado para um contexto académico, existindo, contudo, margem para futuras otimizações em ambientes de produção.

13. Análise Crítica

13.1 Aspetos Positivos

1. Separação de Responsabilidades

A arquitetura distribuída, composta pelo servidor central, pelos servidores de infraestrutura, pelo serviço UDDI e pelo servidor Kerberos, permitiu uma clara separação de responsabilidades, facilitando a manutenção, a evolução e a escalabilidade do sistema.

2. Utilização de Tecnologias Padronizadas

A adoção de **JAX-WS** para a implementação dos Web Services SOAP e do protocolo **Kerberos V5** para autenticação proporcionou uma solução robusta, interoperável e baseada em tecnologias amplamente reconhecidas.

3. Tolerância a Falhas

A implementação da replicação de dados através do protocolo **Quorum Consensus** permitiu assegurar a continuidade do serviço mesmo perante falhas parciais dos servidores.

4. Cobertura Funcional

As principais funcionalidades previstas no enunciado — publicação de anúncios, receção de mensagens, autenticação de utilizadores e gestão das infraestruturas — foram implementadas com sucesso.

5. Integração de Conhecimentos

O desenvolvimento do projeto permitiu integrar conhecimentos adquiridos em diversas unidades curriculares, nomeadamente **Sistemas Distribuídos**, **Segurança Informática**,

Programação de Servidores e Desenvolvimento de Aplicações Móveis, proporcionando uma visão abrangente do desenvolvimento de sistemas distribuídos.

13.2 Aspetos Menos Conseguidos

1. Implementação do Wi-Fi Direct

A implementação da comunicação descentralizada através de **Wi-Fi Direct** revelou algumas limitações, sobretudo ao nível da descoberta automática de dispositivos e da estabilidade das ligações entre equipamentos de fabricantes distintos.

2. Subestimação da Complexidade

A implementação dos mecanismos de tolerância a falhas e da replicação baseada em **Quorum Consensus** revelou-se significativamente mais complexa do que o inicialmente previsto, provocando atrasos em algumas fases do projeto.

3. Segurança da Aplicação Móvel

Embora tenham sido implementados mecanismos de autenticação e proteção das comunicações, aspetos como a cifragem local dos dados e o armazenamento seguro das credenciais poderiam ser aprofundados.

4. Testes em Múltiplos Dispositivos

Os testes foram realizados essencialmente num único dispositivo físico e em emuladores Android, não tendo sido possível validar o comportamento da aplicação em múltiplos dispositivos reais de forma simultânea, exceto através de simulação.

5. Roteamento por Retransmissão

A funcionalidade de roteamento por retransmissão de mensagens, prevista para o funcionamento em modo descentralizado, não chegou a ser implementada devido às limitações de tempo.

15.3 Lições Aprendidas

1. Importância do Planeamento

Uma análise de riscos mais aprofundada na fase inicial teria permitido estimar com maior precisão a complexidade associada aos mecanismos de segurança, replicação e tolerância a falhas.

2. Comunicação da Equipa

A comunicação regular entre os elementos da equipa, aliada à utilização de ferramentas como **Trello** e **GitHub**, revelou-se fundamental para garantir o acompanhamento das tarefas e a coordenação do trabalho.

3. Prototipagem da Interface

A utilização do **Figma** durante a fase de prototipagem permitiu validar antecipadamente a interface da aplicação móvel, reduzindo o retrabalho durante a implementação.

4. Testes Contínuos

A realização sistemática de testes unitários e de integração permitiu identificar e corrigir problemas numa fase precoce do desenvolvimento, contribuindo para a estabilidade do sistema.

5. Documentação

A documentação produzida ao longo do projeto — incluindo diagramas, manuais de utilização e o presente relatório — constitui um recurso essencial para facilitar a compreensão, manutenção e evolução futura do sistema.

14. Conclusão

O projecto **AnunciosLoc** constituiu uma experiência académica enriquecedora, permitindo aplicar, de forma integrada, os conhecimentos adquiridos nas unidades curriculares de **Sistemas Distribuídos, Segurança Informática, Programação de Servidores e Desenvolvimento de Aplicações Móveis**.

Entre as principais realizações do projeto destacam-se:

- A implementação de um sistema distribuído funcional, composto por um servidor central, múltiplos servidores de infraestrutura, autenticação baseada no protocolo Kerberos e descoberta dinâmica de serviços através do UDDI.
- O desenvolvimento de uma aplicação móvel para Android, com suporte para captura de localização GPS, comunicação baseada em Web Services SOAP e persistência local dos dados.
- A implementação de mecanismos de tolerância a falhas através da replicação de dados e do protocolo **Quorum Consensus**.
- O desenvolvimento de uma aplicação Web destinada à administração e monitorização do sistema.

Durante o desenvolvimento do projeto foram igualmente identificados alguns desafios importantes, nomeadamente:

- A complexidade inerente à integração de múltiplos servidores e protocolos, como **SOAP, Kerberos e UDDI**.
- A instabilidade da tecnologia **Wi-Fi Direct** na comunicação descentralizada entre dispositivos móveis.
- A subestimação do esforço necessário para implementar os mecanismos de replicação, consistência dos dados e segurança do sistema.

Em síntese, o **AnunciosLoc** atingiu os objetivos inicialmente propostos, disponibilizando uma plataforma funcional para a publicação e receção de anúncios baseados na localização geográfica dos utilizadores. A arquitetura adotada revelou-se flexível, modular e escalável, permitindo a evolução futura do sistema através da incorporação de novas funcionalidades e otimizações.

Para além dos resultados obtidos, o desenvolvimento deste projeto contribuiu significativamente para a consolidação dos conhecimentos práticos da equipa no domínio dos sistemas distribuídos, constituindo uma base sólida para futuros trabalhos de investigação e desenvolvimento.

15. Anexos

15.1 Guia de Instalação e Execução

Pré-requisitos

Antes da instalação do sistema, devem estar disponíveis os seguintes componentes:

- Java JDK 8 ou superior;
- Apache Maven 3.6 ou superior;
- PostgreSQL 12 ou superior;
- Android Studio (para compilação da aplicação móvel).

Procedimento de Instalação

1. Clonar o repositório

```
git clone https://github.com/seu-usuario/anunciosloc.git
```

2. Configurar a base de dados

- Criar uma base de dados denominada anunciosloc.
- Configurar o utilizador e a palavra-passe da base de dados no ficheiro pom.xml do projeto anunciosloc-ws.

3. Compilar o projeto

```
cd anunciosloc
```

```
mvn clean install -DskipTests
```

4. Iniciar os servidores

- Servidor **AnunciosLoc**:
- `cd anunciosloc-ws && mvn exec:java ...`

- Servidores de infraestrutura (executar em três terminais distintos):
- `cd infrastructure-ws && mvn exec:java ...`
- Servidor **Kerberos**:
- `cd kerberos-ws && mvn exec:java ...`

5. Executar a aplicação móvel

- Abrir o projeto no Android Studio.
- Compilar a aplicação.
- Instalar a aplicação num dispositivo Android ou num emulador.

6. Executar a aplicação Web

Abrir o ficheiro `anunciosloc-web/index.html` num navegador Web.

15.2 Comandos para Execução dos Servidores

Servidor Central (porta 8080)

`cd anunciosloc/anunciosloc-ws`

`mvn exec:java '-Dexec.mainClass=pt.uan.anunciosloc.server.AnunciosLocServer'`

Infraestrutura 1 (porta 8081)

`cd anunciosloc/infrastructure-ws`

`mvn exec:java '-Dexec.args=Largo_Independencia -8.8167 13.2344 20 50 8081'`

Infraestrutura 2 (porta 8082)

`cd anunciosloc/infrastructure-ws`

`mvn exec:java '-Dexec.args=Belas_Shopping -8.9012 13.1876 30 200 8082'`

Infraestrutura 3 (porta 8083)

`cd anunciosloc/infrastructure-ws`

`mvn exec:java '-Dexec.args=Ginasio_Camama -8.9234 13.2100 15 50 8083'`

```
# Servidor Kerberos (porta 8085)
```

```
cd anunciosloc/kerberos-ws
```

```
mvn exec:java '-Dexec.mainClass=pt.uan.anunciosloc.kerberos.KerberosServer'
```

15.3 Configuração da Base de Dados

O sistema utiliza uma base de dados PostgreSQL para armazenar toda a informação relativa aos utilizadores, anúncios, infraestruturas, réplicas, sessões e entregas.